

Lab 04 — Prime Path Coverage

1. Objectives

By the end of this lab you will be able to:

1. Build a control flow graph from a piece of C++ code that contains nested loops, a two-way branch, and a switch.
2. Design GoogleTest cases that give node coverage and then edge coverage of that graph.
3. Tour a supplied set of prime paths and recognise when a prime path is infeasible.

2. Learning outcomes

- **LO1** — Number the nodes and edges of a CFG that mixes loops with straight-line branching.
- **LO2** — Explain why edge coverage is stronger than node coverage, and demonstrate it with tests.
- **LO3** — Read a prime path off the graph, write a grid that tours it, and argue why three of the listed prime paths cannot be toured.

3. The unit under test

You are given *src/region_scan.hpp* and *src/region_scan.cpp* files in your starter code. The actual lab objective is to write test cases in the file *test/lab04_tests.cpp*.

scanRegion walks a rectangular region of integer cells, adds the cells up, and turns that total into a reported score. The switch hands the score to a second function, *adjust*, which clamps it into the band [0, 100].

```
int scanRegion(const Grid& grid) {
    int sum = 0;

    for (int r = 0; r < (int)grid.size(); ++r) {
        for (int c = 0; c < (int)grid[r].size(); ++c) {
            sum += grid[r][c];
        }
    }

    int score;
    if (sum > 10) {
        score = sum;
    } else {
        score = -sum;
    }

    switch (sum % 3) {
        case 0:
```

```

        score = adjust(score);
        break;
    default:
        break;
}

return score;
}

```

4. The control flow graph

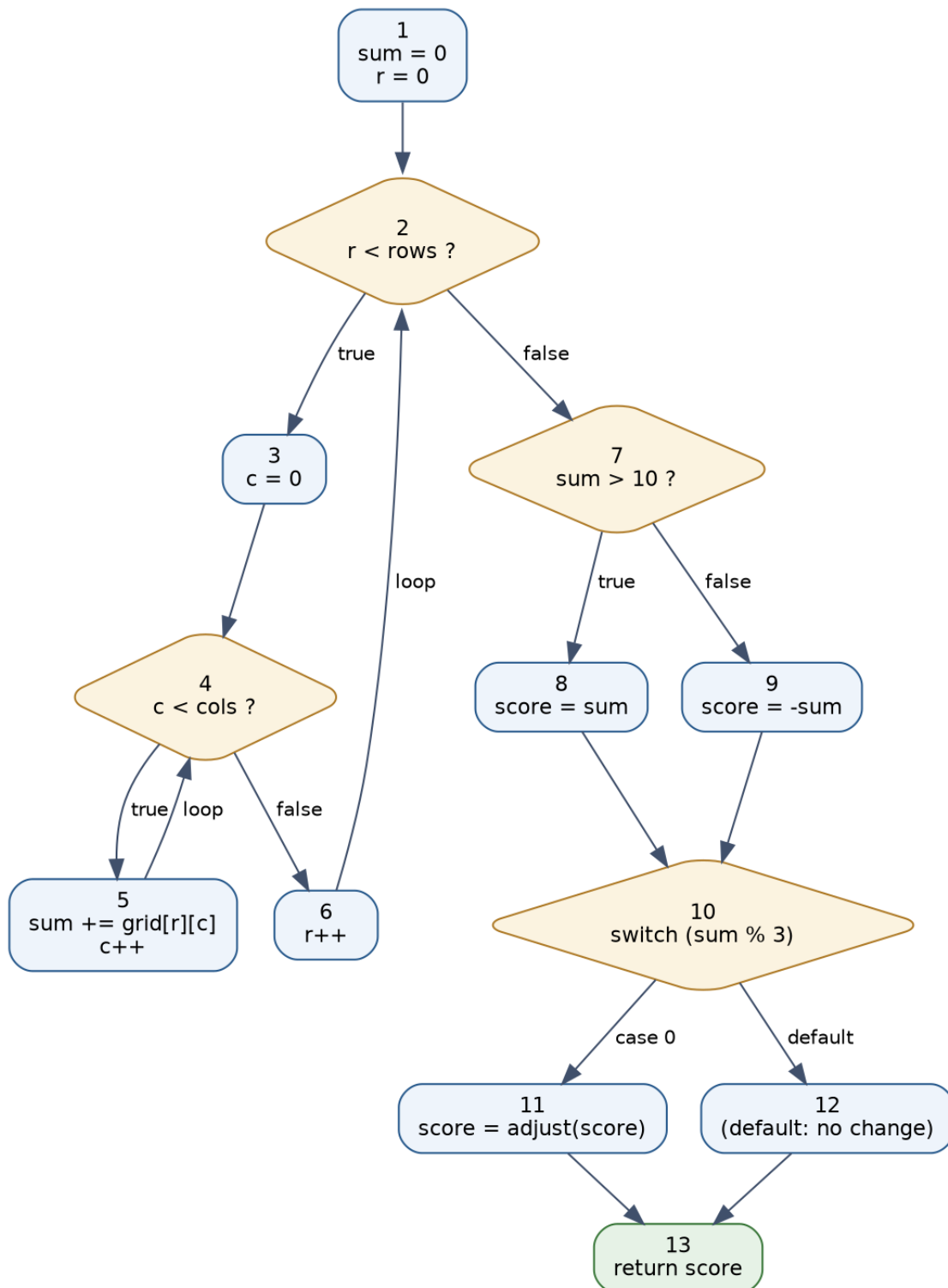
The graph has thirteen nodes. Node 1 is the initial node; node 13 (the return) is the final node. The three decision nodes are the outer loop test (2), the inner loop test (4), and the two-way branch (7); the switch is node 10.

Node	Statement
1	sum = 0; r = 0
2	r < rows ?
3	c = 0
4	c < cols ?
5	sum += grid[r][c]; c++
6	r++
7	sum > 10 ?
8	score = sum
9	score = -sum
10	switch (sum % 3)
11	score = adjust(score)
12	default: no change
13	return score

The edge set is:

(1,2) (2,3) (2,7) (3,4) (4,5) (4,6) (5,4) (6,2)
 (7,8) (7,9) (8,10) (9,10) (10,11) (10,12) (11,13) (12,13)

Control flow graph - scanRegion()



Control flow graph of scanRegion

5. Prime paths

There are twenty-two prime paths in this example. Ideally, you should tour the most feasible paths (as mentioned in Task 4) and focus on writing test cases for these.

P01 4-5-4
P02 5-4-5
P03 1-2-3-4-5
P04 1-2-3-4-6
P05 2-3-4-6-2
P06 3-4-6-2-3
P07 4-6-2-3-4
P08 5-4-6-2-3
P09 6-2-3-4-5
P10 6-2-3-4-6
P11 1-2-7-8-10-11-13 (infeasible)
P12 1-2-7-8-10-12-13 (infeasible)
P13 1-2-7-9-10-11-13
P14 1-2-7-9-10-12-13 (infeasible)
P15 3-4-6-2-7-8-10-11-13
P16 3-4-6-2-7-8-10-12-13
P17 3-4-6-2-7-9-10-11-13
P18 3-4-6-2-7-9-10-12-13
P19 5-4-6-2-7-8-10-11-13
P20 5-4-6-2-7-8-10-12-13
P21 5-4-6-2-7-9-10-11-13
P22 5-4-6-2-7-9-10-12-13

6. Tasks

1. **Draw the CFG.** Reproduce the graph from the node and edge tables above. Number every node and label the decision edges with the condition that takes them (for example true / false on node 7). Save it as diagrams/cfg.png in your submission. A photo of a neat hand drawing is fine.
2. **Node coverage.** Write GoogleTest cases so that, between them, every node is executed at least once. State in your report which grid reaches nodes 8 and 11 and which reaches nodes 9 and 12.
3. **Edge coverage.** Add the tests needed to cover the edges that node coverage leaves out. One of them, edge (2,7), can only be taken by a region the outer loop never enters.
4. **Prime path tour.** Using the list in Section 5, add tests whose executions tour the nineteen *feasible prime paths*. Label each test with the prime path numbers it covers, the way the worked example is labelled. Do not write tests for P11, P12, or P14; instead explain their infeasibility in your report.

Build and run at each step:

```
cmake -S . -B build
cmake --build build --target lab04_tests
./build/lab04_tests --gtest_color=yes
```

7. Standard submission

1. Follow the same submission format (as discussed previously).
2. The submission deadline is Tuesday, 6 October 2026, 09:00 AM.